

# Assignment 06 (Solution)

## Exercise 2:

1. Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of registers x13 and x14 be?

```
1      addi x11, x12, 5
2      add x13, x11, x12
3      addi x14, x11, 15
```

x11 = 27

x13 = 33

x14 = 26

2. Consider the following code. What would the final values of register x15 be? Assume the register file is written at the beginning of the cycle and read at the end of the cycle. Therefore, an ID stage will return the results of a WB state occurring during the same cycle.

```
1      addi x11, x12, 5
2      add x13, x11, x12
3      addi x14, x11, 15
4      add x15, x11, x11
```

x11 = 27

x13 = 33

x14 = 26

x15 = 54

The code will run correctly because the result of the first instruction is written back to the register file at the beginning of the 5th cycle, whereas the final instruction reads the updated value of x1 during the second half of this cycle.)

3. Add NOP instructions to the code below so that it will run correctly on a pipeline that does not handle data hazards.

```
1      addi x11, x12, 5
2      add x13, x11, x12
3      addi x14, x11, 15
4      add x15, x13, x12
```

Code with NOP :

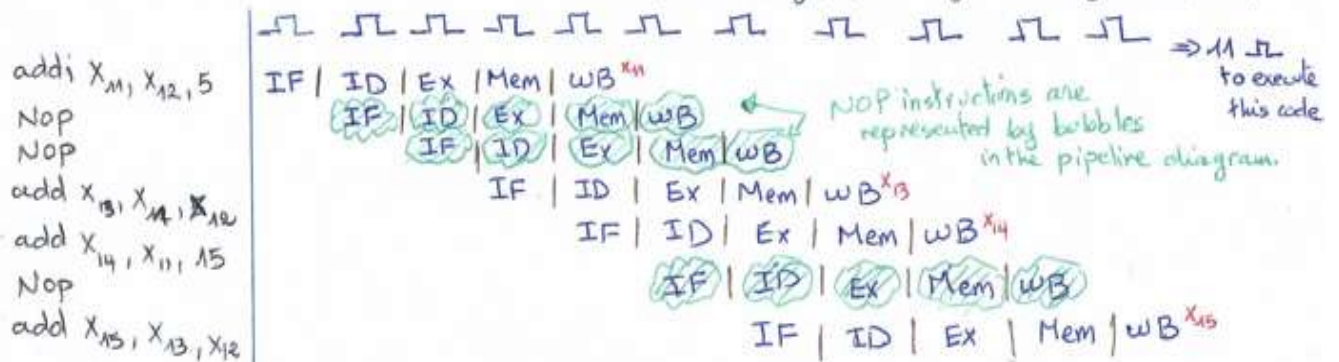
```

1  addi x11, x12, 5
2  NOP
3  NOP
4  add x13, x11, x12
5  addi x14, x11, 15
6  NOP
7  add x15, x13, x12

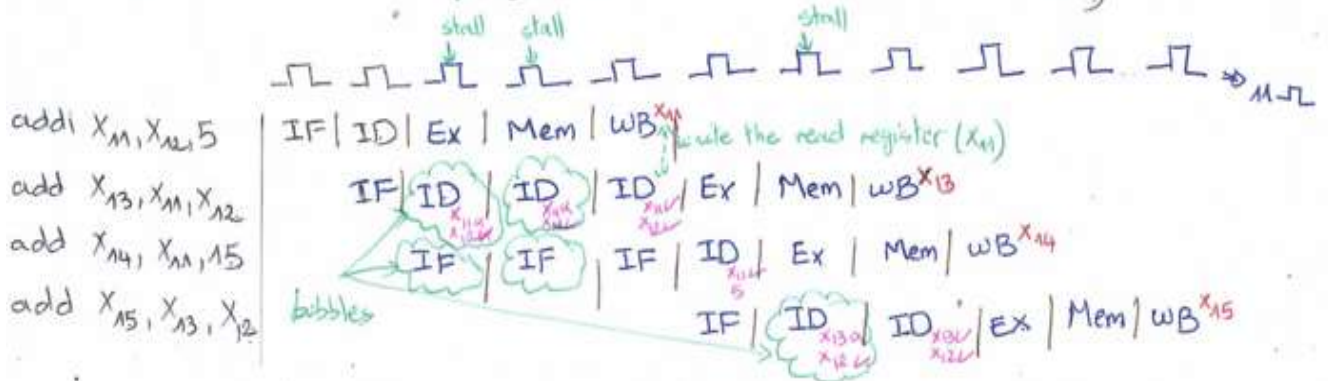
```

Explanation :

① stalling by adding Nop (compiler & software solution)  
 detects data hazards and fix them by adding Nop.



② stalling without insert Nop (Hazard Detection Unit & hardware solution)



Note: The second solution is an improvement of the first one (more used in the modern pipelined CPU if the forwarding (optimal and best solution) cannot be used)

4. Suppose that (after optimization) a typical  $n$ -instruction program requires an additional  $4 \times n$  NOP instructions to correctly handle data hazards.
- Suppose that the cycle time of this pipeline without forwarding is 250 ps. Suppose also that adding forwarding hardware will reduce the number of NOPs from  $4 \times n$  to  $.05 \times n$ , but increase the cycle time to 300 ps. What is the speedup of this new pipeline compared to the one without forwarding?
  - Different programs will require different amounts of NOPs. How many NOPs (as a percentage of code instructions) can remain in the typical program before that program runs slower on the pipeline with forwarding?
  - Repeat b; however, this time let  $x$  represent the number of NOP instructions relative to  $n$ . (In b,  $x$  was equal to .4.) Your answer will be with respect to  $x$ .
  - Can a program with only  $.075 \times n$  NOPs possibly run faster on the pipeline with forwarding? Explain why or why not.
  - At minimum, how many NOPs (as a percentage of code instructions) must a program have before it can possibly run faster on the pipeline with forwarding?

Solution :

- The pipeline without forwarding requires  $1.4 \times n \times 250ps$ . The pipeline with forwarding requires  $1.05 \times n \times 300ps$ . The speedup is therefore  $(1.4 \times 250)/(1.05 \times 300) = 1.11$ .
- Our goal is for the pipeline with forwarding to be faster than the pipeline without forwarding. Let  $y$  be the number of stalls remaining as a percentage of "code" instructions. Our goal is for  $300 \times (1 + y) \times n < 250 \times 1.4 \times n$ . Thus,  $y$  must be less than 16.7%.
- This time, our goal is for  $300(1 + y) \times n < 250(1 + x) \times n$ . This happens when  $y < (250x - 50)/300$ .
- It cannot. In the best case, where forwarding eliminates the need for every NOP, the program will take time  $300 \times n$  to run on the pipeline with forwarding. This is slower than the  $250 \times 1.075 \times n$  required on the pipeline with no forwarding.
- Speedup is not possible when the solution to c is less than 0. Solving  $0 < (250x - 50)/300$  for  $x$  gives that  $x$  must be at least 0.2.

### Exercise 3:

Consider the following loop.

```

1 LOOP: lw x10, 0(x13)
2       lw x11, 8(x13)
3       add x12, x10, x11
4       addi x13, x13, 16
5       bnez x12, LOOP

```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, that the pipeline has full forwarding support, and that branches are resolved in the EX (as opposed to the ID) stage.

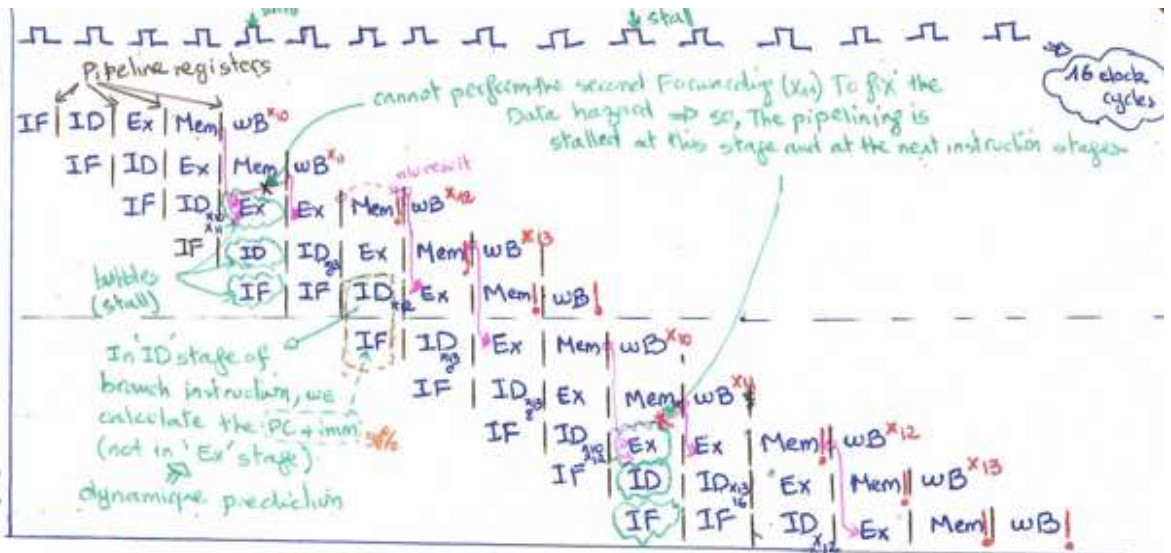
- Show a pipeline execution diagram for the first two iterations of this loop.
- Mark pipeline stages that do not perform useful work. How often while the pipeline is full do we have a cycle in which all five pipeline stages are doing useful work? (Begin with the cycle during which the `addi` is in the IF stage. End with the cycle during which the `bnez` is in the IF stage.)



Loop:

Lw  $x_{10}, 0(x_{13})$   
 Lw  $x_{11}, 8(x_{13})$   
 add  $x_{12}, x_{10}, x_{11}$   
 addi  $x_{13}, x_{13}, 16$   
 bnez  $x_{12}, \text{Loop}$

Lw  $x_{10}, 0(x_{13})$   
 Lw  $x_{11}, 8(x_{13})$   
 add  $x_{12}, x_{10}, x_{11}$   
 addi  $x_{13}, x_{13}, 16$   
 bnez  $x_{12}, \text{Loop}$



2). The pipelined stages that do not perform useful work are marked by '!' in the above pipeline execution diagram.

• In a particular clock cycle, a pipeline stage is not doing useful work if it is stalled or if the instruction going through that stage is not doing any useful work there.

As the diagram above shows, there are not any cycles during which every pipeline stage is doing useful work.